CSC3100: Priority Queue & Heap

Tianshu Yu

2024 Fall

Motivation

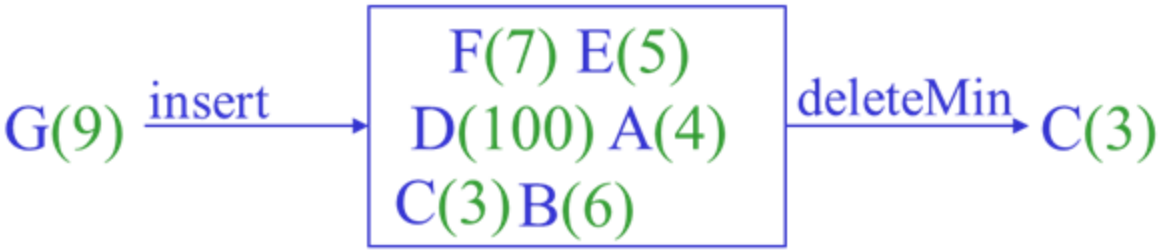
- Have you ever been jammed by a huge job while you are waiting for just one-page printout ?
- This is a typical situation for a simple first-in first-out (FIFO) queue
- Can there be a “smarter” printer?

Motivation (Cont.)

- Other applications
  - Scheduling CPU jobs
  - Emergency room admission processing
- Problems using FIFO queue
  - Short jobs may go first
  - Most urgent cases should go first
  - Task with highest priority/lowest priority should go first

Priority Queue

- Priority Queue operations
  - insert
  - deleteMin
  - create
  - isEmpty
  - …



- Priority Queue property: for two elements in the queue, ***x*** and ***y***, if ***x*** has a ***lower priority*** value than ***y***, ***x*** will be deleted before ***y***

Simple Implementations

- There are multiple possibilities for the implementation.
- Simple linked list (Suggestion 1)
  - Insert at the front in O(1)
  - Delete minimum in O(*N*)
- Sorted linked list (Suggestion 2)
  - Insert in O(*N*)
  - Delete minimum in O(1)
- Sorted Array (Suggestion 3)
  - Insert in O(*N*)
  - Delete minimum in O(*N*)

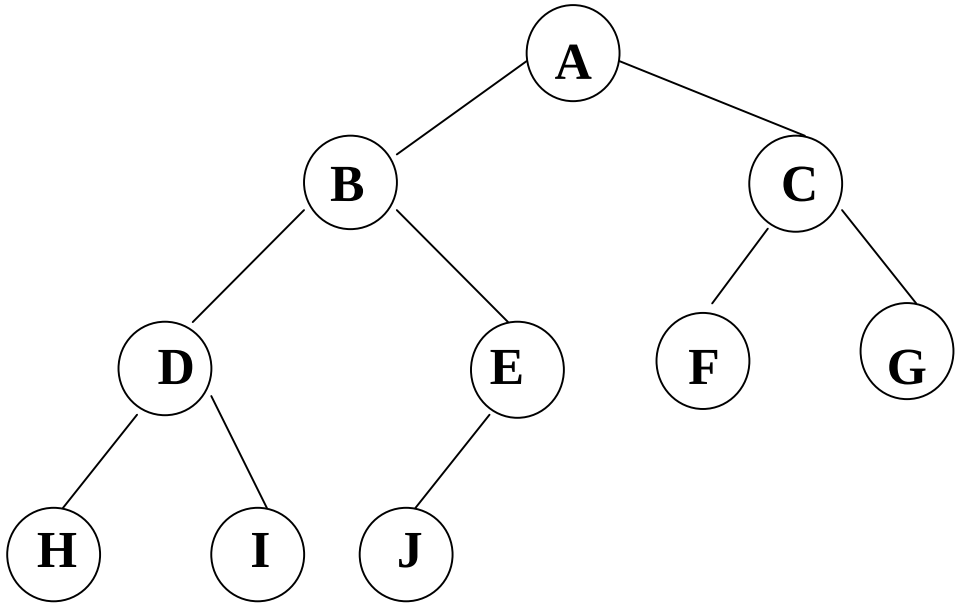
Simple Implementations

- Binary heap (Best solution)
  - O (log *N*) for insert and deleteMin for worst cases
  - Two properties:
    - Structure property
    - Heap order property
- Other solution: binary search tree (to be explained later)
  - O (log *N*) on average for both operations

Binary Heap (Heap)

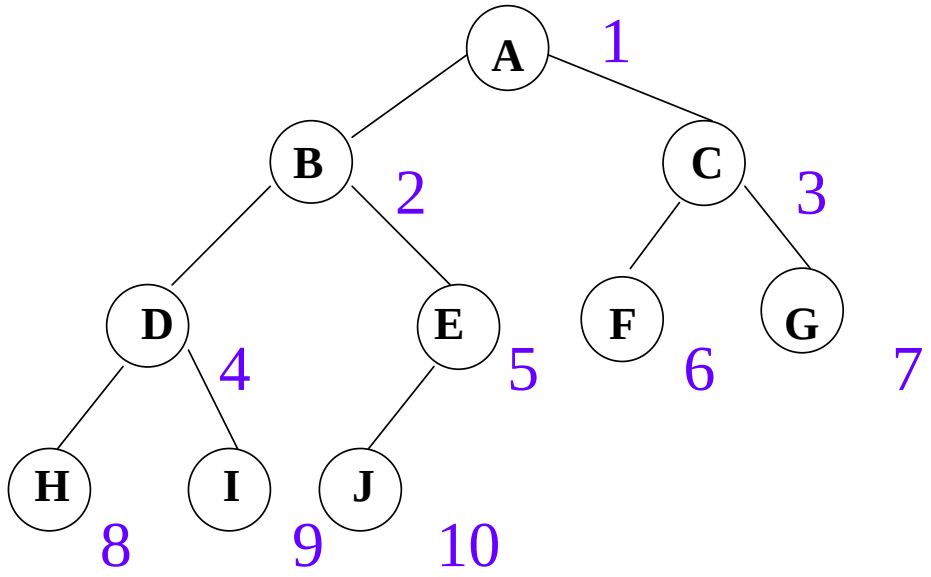
- (A) Structure Property
  - A heap is a binary tree that is completely filled, except at the bottom level, which is filled from left to right.
  - A complete binary tree of height h has between 2^h and $2^{h+1} - 1$ nodes
  - The height of a complete binary tree $= \lfloor \log N \rfloor$ (round down, e.g. $\lfloor 2.7 \rfloor = 2$).

Binary Heap (Heap)



- A complete binary tree can be represented in an array without using pointers.

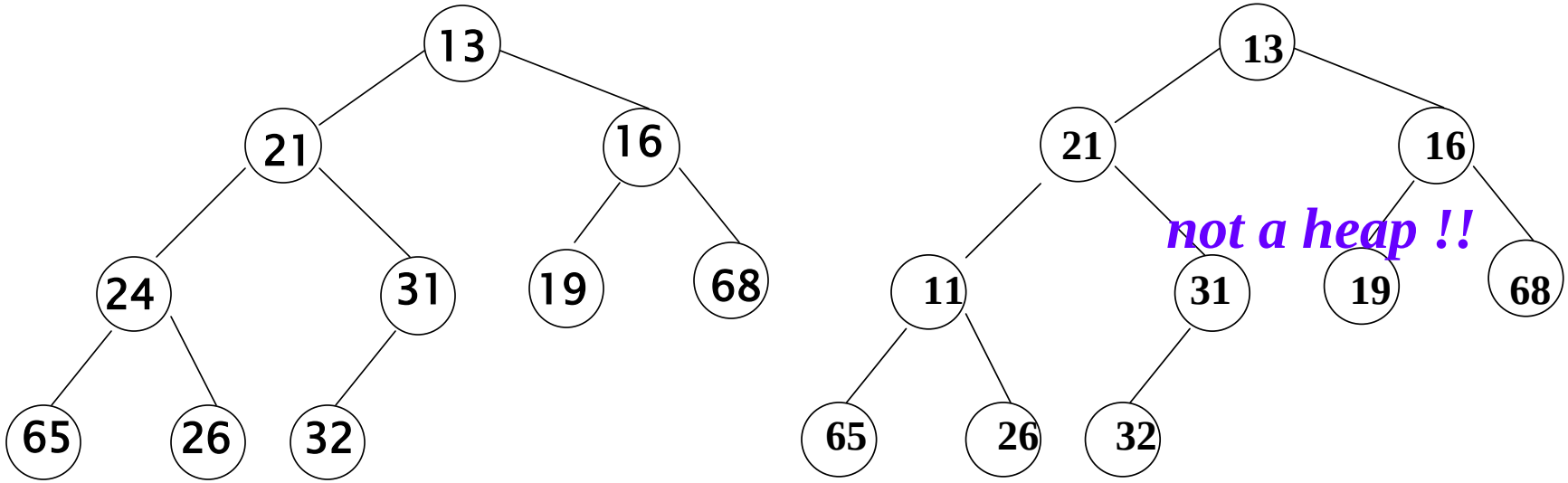
	A	B	C	D	E	F	G	H	I	J					
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

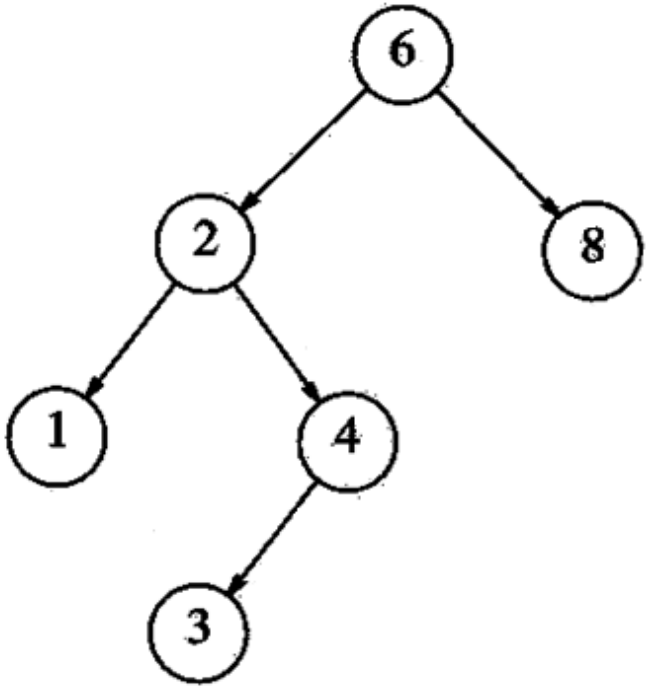


- The root is at position 1 (reserve position 0 for the *implementation purpose*).
- For an element at position i, its left child is at position $2i$ and its right child at $2i+1$; its parent is at floor $\lfloor i/2 \rfloor$.

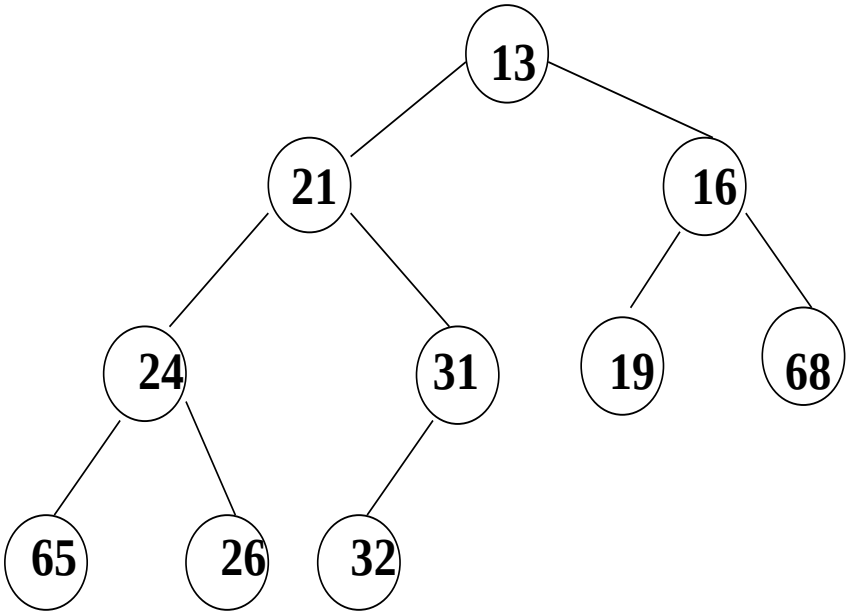
Binary Heap

- (B) Heap Order Property
  - The value at any node should be smaller than (or equal to) all of its descendants (guarantee that the node with the minimum value is at the root).





A Binary Search Tree



A Heap

Note the difference in node ordering!!

Binary Heap

- Class skeleton for **Elements**

```
class ElementType {
        int priority;
        String data;
        public ElementType(int priority, String data) {
                this.priority = priority;
                this.data = data;
        }
        public boolean isHigherPriorityThan(ElementType e) {
                return priority < e.priority;
        }
}
```

Binary Heap

- Definition and constructor of Priority Queue

```
public class BinaryHeap {
        private int currentSize;          // Number of elements in heap
        private ElementType[] arr;    // The heap array

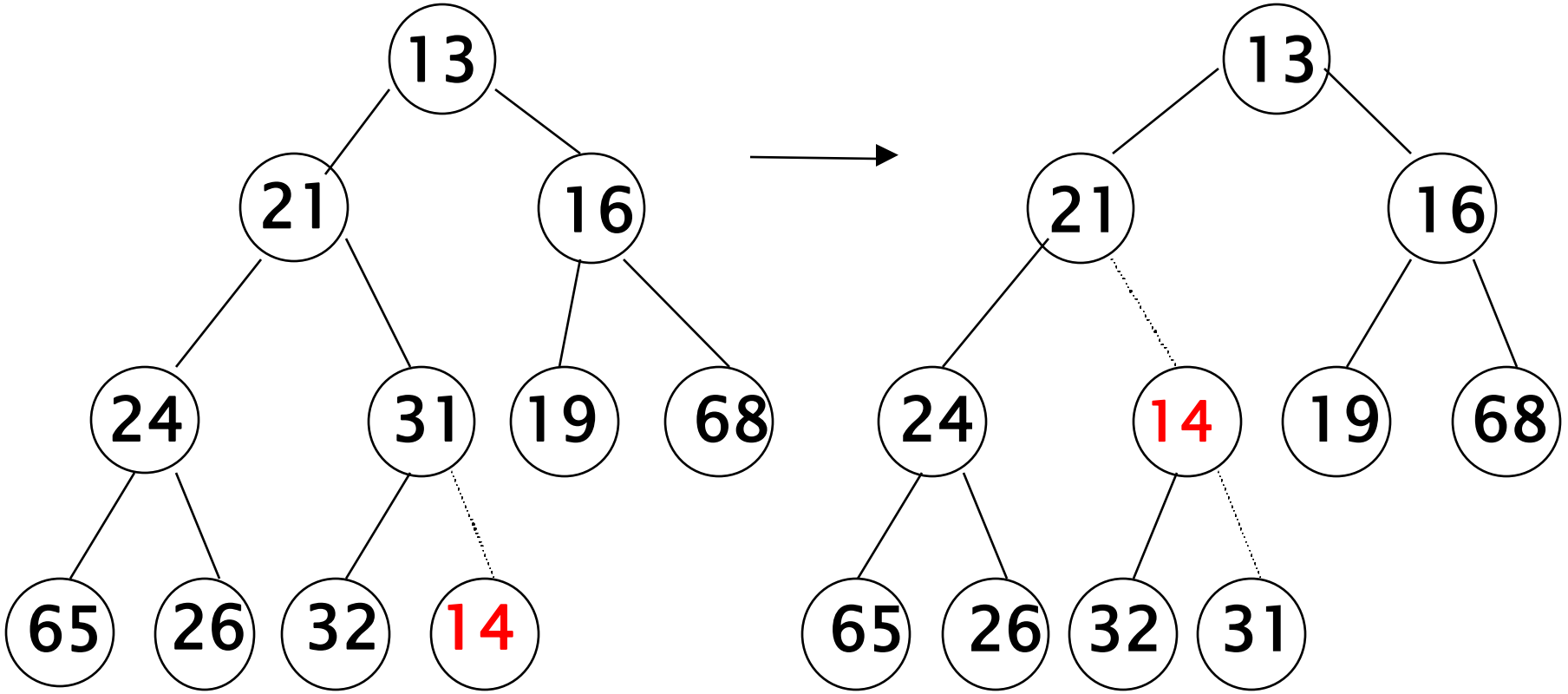
        public BinaryHeap (int capacity) {
                currentSize = 0;
                arr = new ElementType[capacity + 1];
        }
}
```

Binary Heap

- Some functions
  - void destroy ( );
  - void makeEmpty ( );
  - void insert (ElementType X);
  - ElementType deleteMin ( );
  - ElementType findMin ( );
  - boolean isEmpty ( );
  - boolean isFull ( );

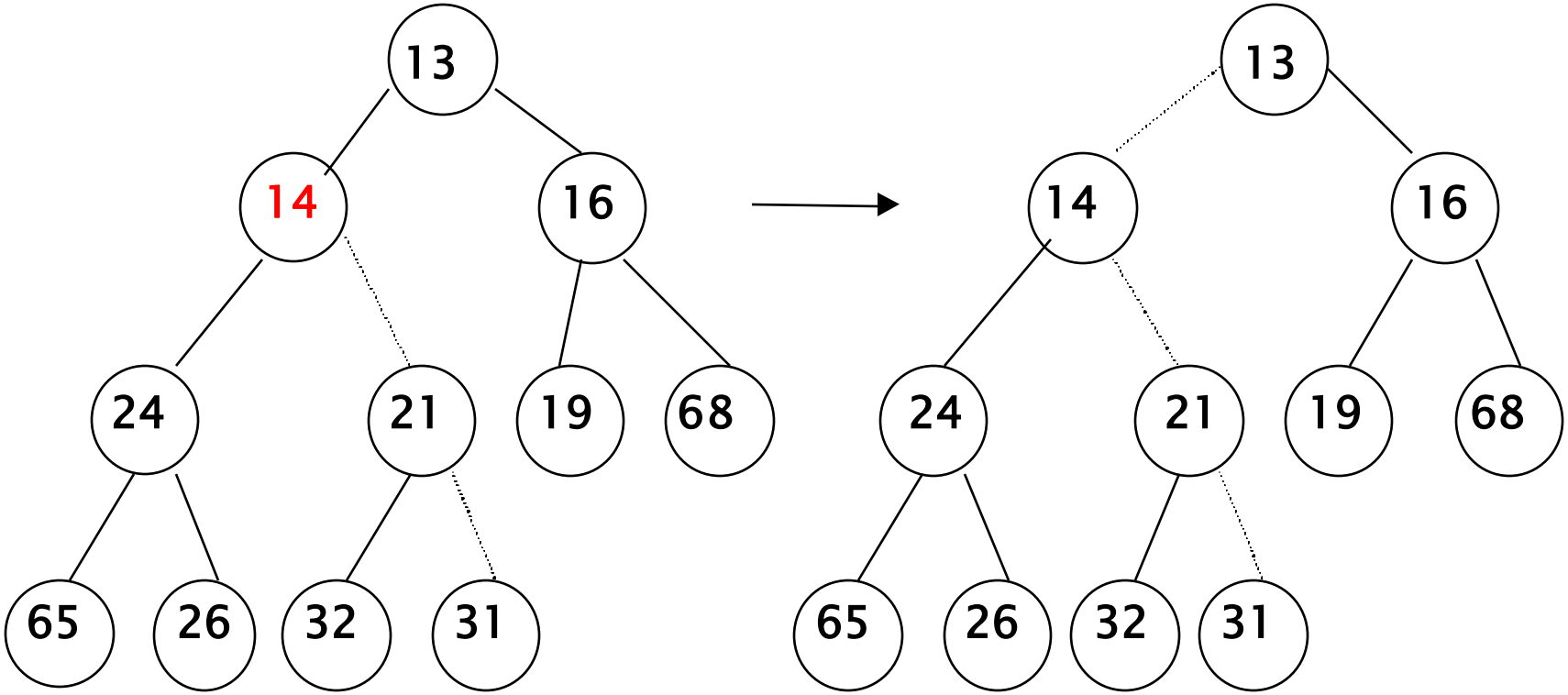
Binary Heap – Insert

Attempt to insert 14:
(1) creating the hole, and (2) bubbling the hole up



Binary Heap – Insert

The remaining two steps to insert 14 in previous heap



Binary Heap – Insert

- To insert an element X,
  - create a hole in the next available location.
  - If X can be placed in the hole without violating heap order, insertion is complete.
  - Otherwise slide the element that is in the hole's parent node into the hole, i.e., bubbling the hole up towards the root.
  - Continue this process until X can be placed in the hole (a *percolating up* process).

Binary Heap – Insert

Attention!

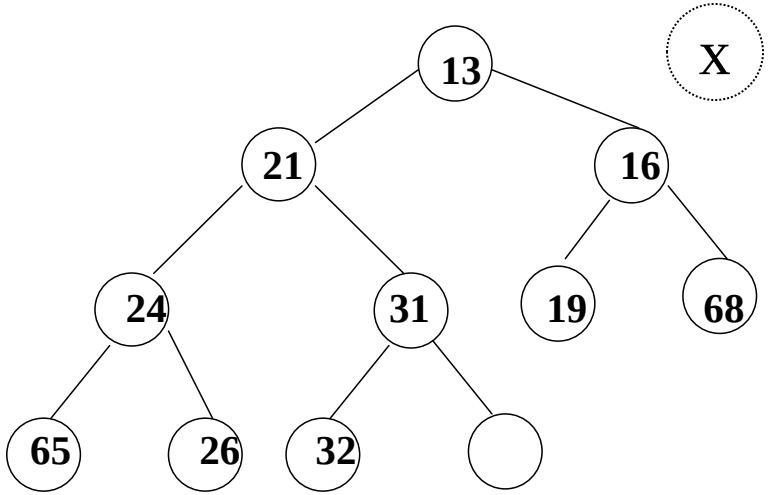
Worst case running time is O (log *N*) – the new element is percolating up all the way to the root.

Question: when does this happen?

Binary Heap – Insert

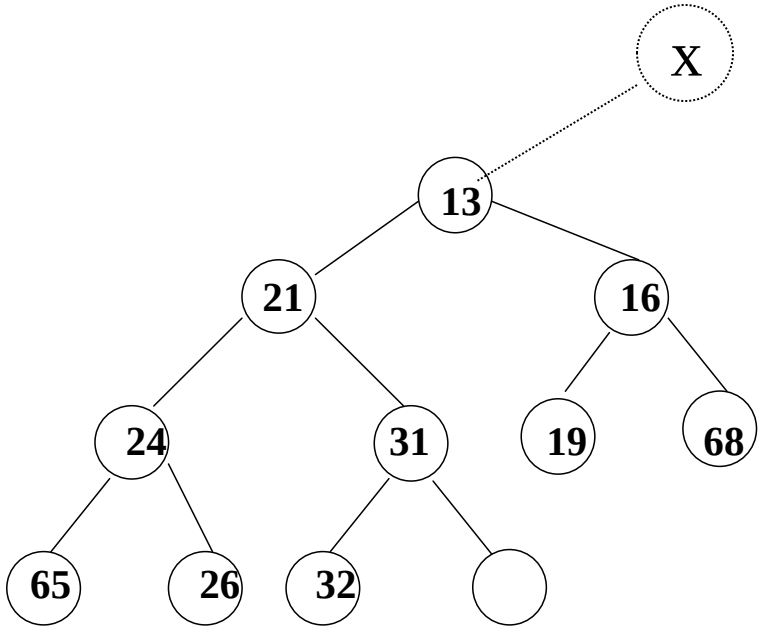
```
public void insert(ElementType x) throws Exception {
        if (isFull())
                throw new Exception("Overflow");

        // Percolate up
        int hole = ++currentSize;
        while(hole > 1 && x.isHigherPriorityThan(arr[hole/2])) {
                arr[hole] = array[hole / 2];
                hole /= 2;
        }
        arr[hole] = x;
}
```



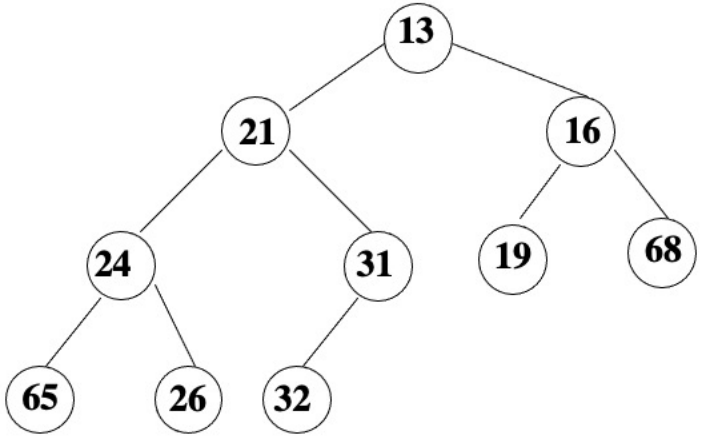
Binary Heap – Insert (another implementation)

```
public void insert(ElementType x) throws Exception {
        if (isFull())
                throw new Exception("Overflow");
        // Percolate up
        int hole = ++currentSize;
        for(arr[0] = x; x.isHigherPriorityThan(arr[hole/2]));hole /= 2)
                arr[hole] = array[hole / 2];
        arr[hole] = x;
}
```



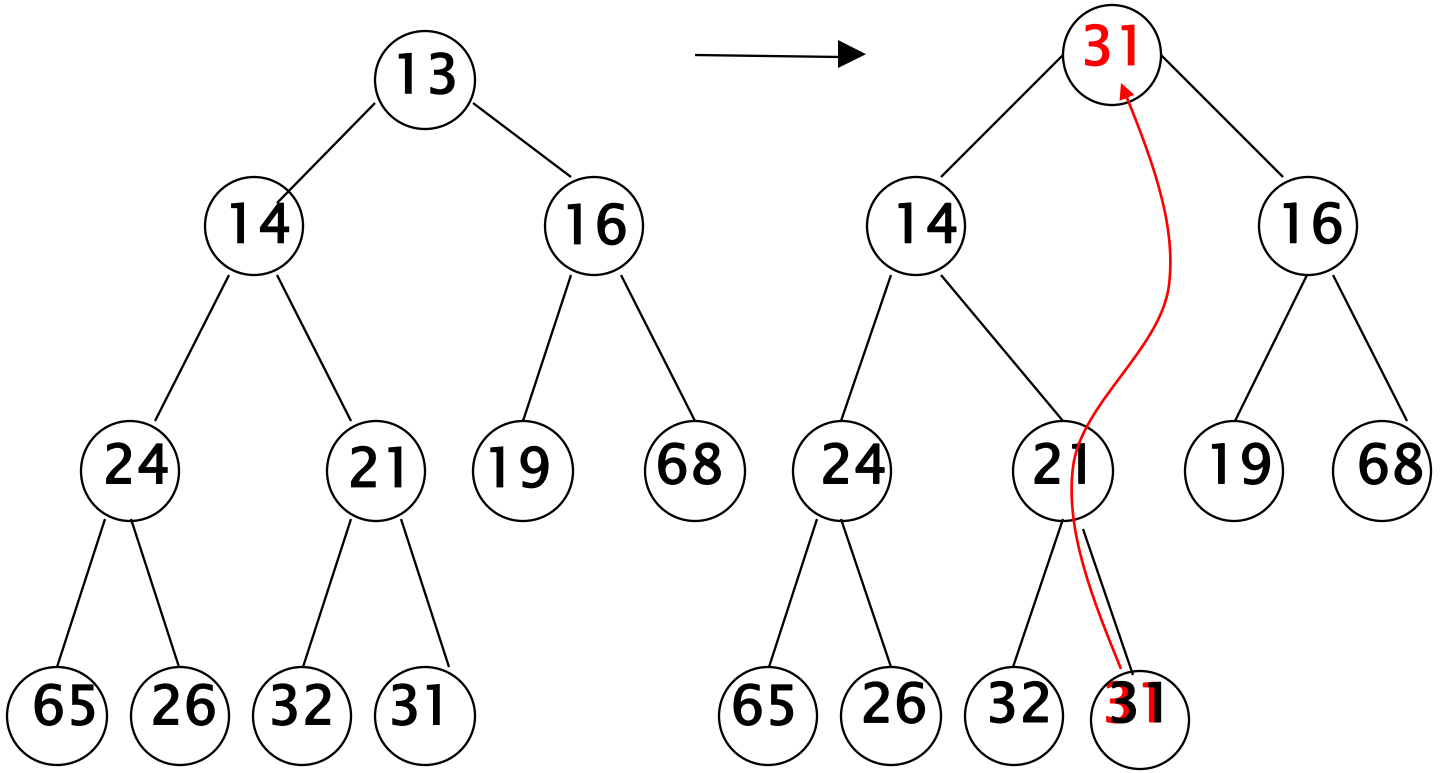
Exercise

- Given a binary heap as shown below, please show the procedure of inserting an element 20 into the heap step by step



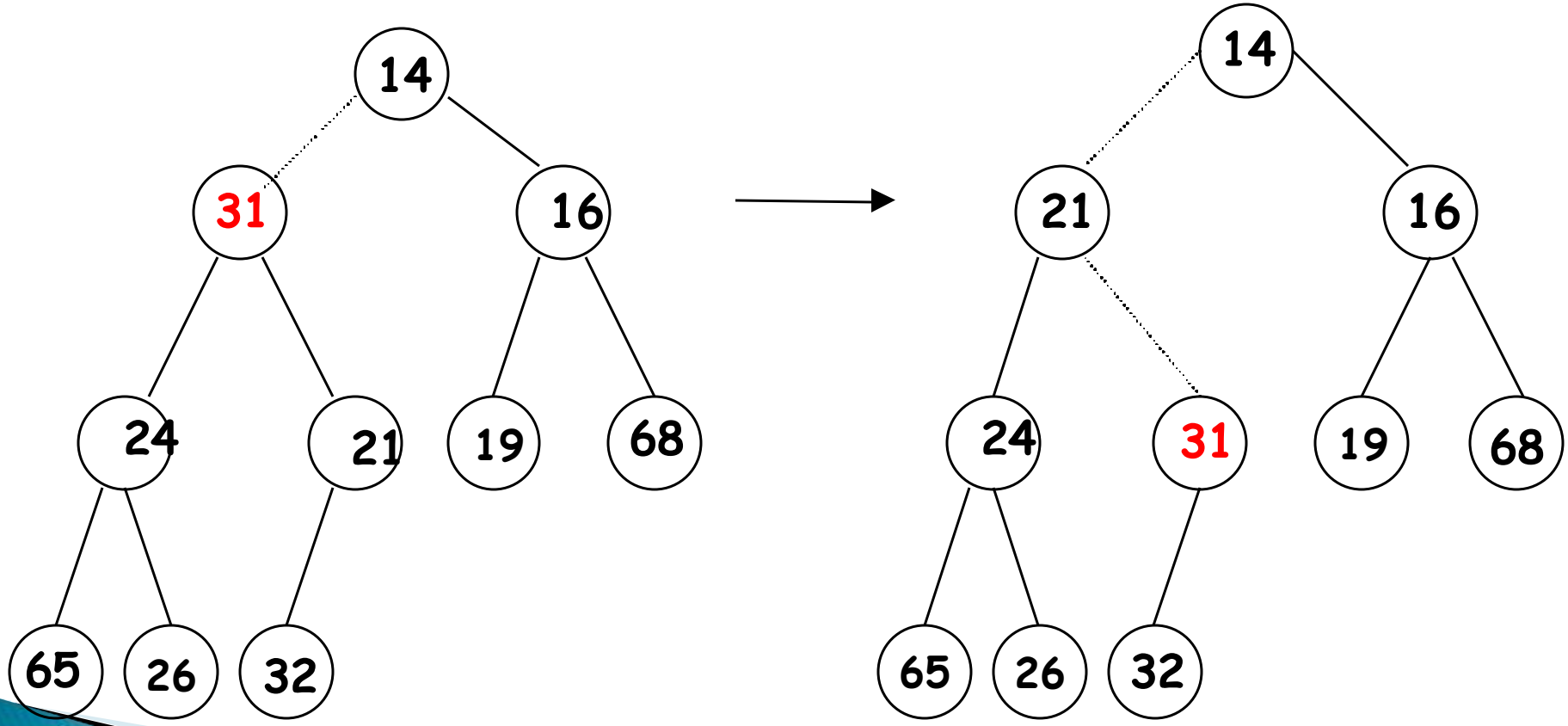
Binary Heap – DeleteMin

Creation of the hole at the root



Binary heap: deleteMin

Next two steps in DeleteMin



Binary Heap – DeleteMin

- The element at the root (position 1) is to be removed, and a hole is created.
- Fill the root with the last node X.
- Percolate X down (switch X with the **smaller** child) until the heap order property is satisfied.

Binary Heap – DeleteMin

- Some node may have only one child (be careful when coding!).
- Worst case running time is O (log *N*).

Binary Heap – DeleteMin

```
public String deleteMin() {
        if (isEmpty())
                return null;

        String data = arr[1].data;
        arr[1] = arr[currentSize--];

        percolateDown(1);
        return data;
}
```

Binary heap: percolateDown

```
private void percolateDown(int hole) {
        int child;
        ElementType tmp = arr[hole];
        while (hole * 2 <= currentSize) {
                child = hole * 2;
                if (child != currentSize &&
                        arr[child +1].isHigherPriorityThan(arr[child]))
                        child++;
                if (arr[child].isHigherPriorityThan(tmp))
                        arr[hole] = array[child];
                else
                        break;
                hole = child;
        }
        arr[hole] = tmp;
}
```

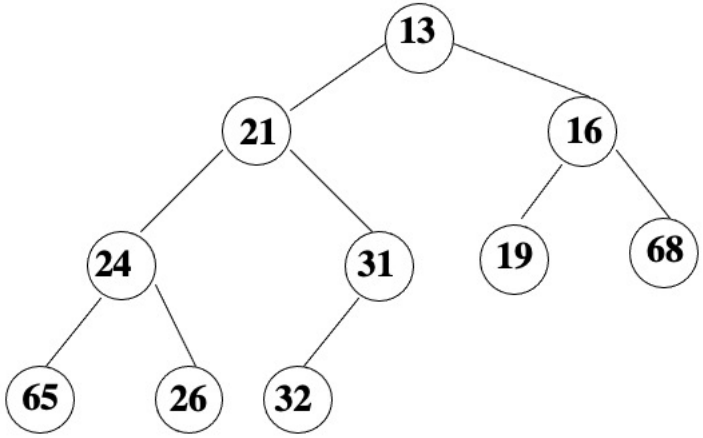
Binary Heap – percolateDown

Consider

- Given a node, assuming its left & right subtrees satisfy heap properties, design a recursive percolateDown() sub-routine.

Exercise

- Given a binary heap as shown below, please show the procedure of deletion on the heap step by step



Binary Heap – Other Heap Operations

decreaseKey (P, Δ)

- Lower the key value at position P by Δ.
- Fix the heap order by percolating up.
- Application: advance the priority of a job.

Complexity analysis

- Given a heap with n elements
  - The height/depth of the heap is $O(\log n)$
    - Why?
  - During insertion/deletion, the worst-case time complexity depends linearly to the height/depth of the heap
- Heap insertion
  - $O(\log n)$
- Heap deletion
  - $O(\log n)$

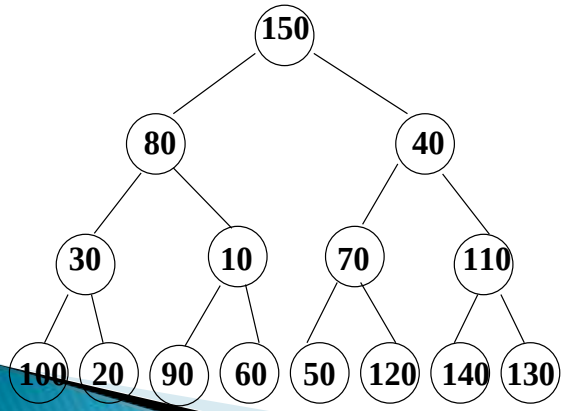
Binary heap construction

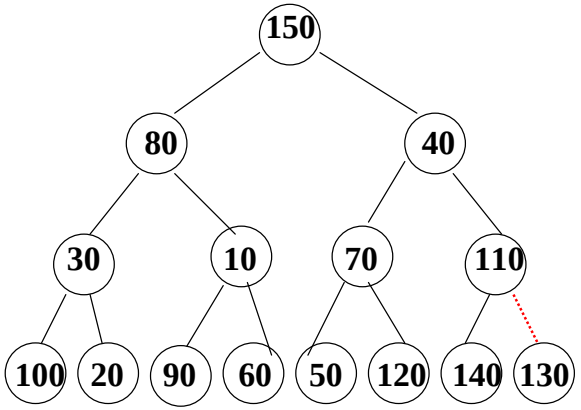
A naïve algorithm to build the binary heap is to repeatedly insert nodes one by one, which completes in O(nlogn) time

A faster algorithm to build the binary heap:

- *N* successive appends at the end of the array, each taking O(1), so the tree is unordered
- for (*i* = *n*/2; i > 0; i--)

  percolateDown (*i*);

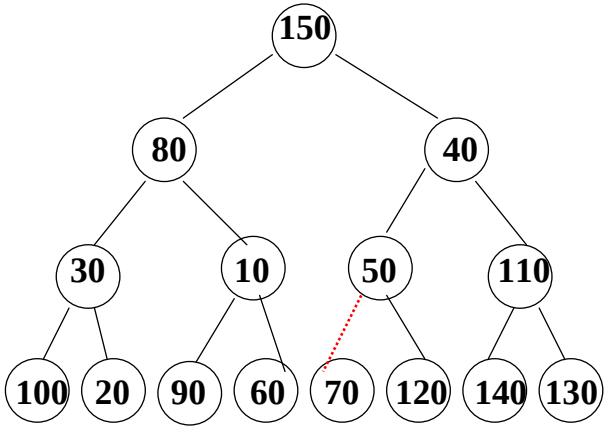




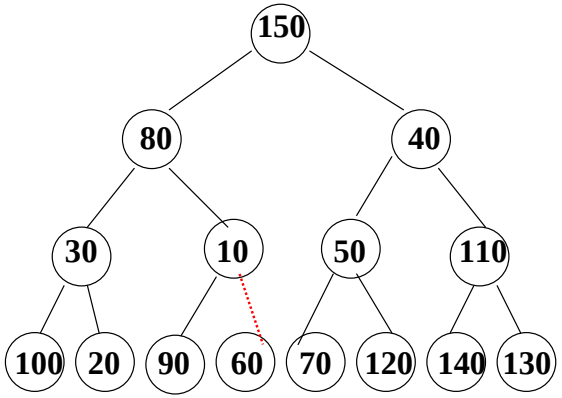
After percolateDown (7)

Note: Each dashed line corresponds to two comparisons: one to find the smaller child, and one to compare the smaller child with the node.

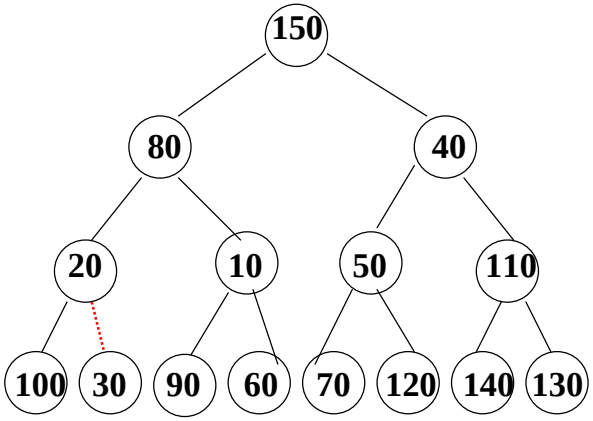
Binary heap construction



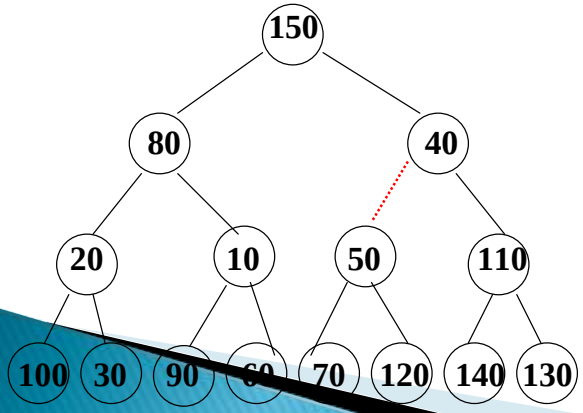
After *percolateDown* (6)



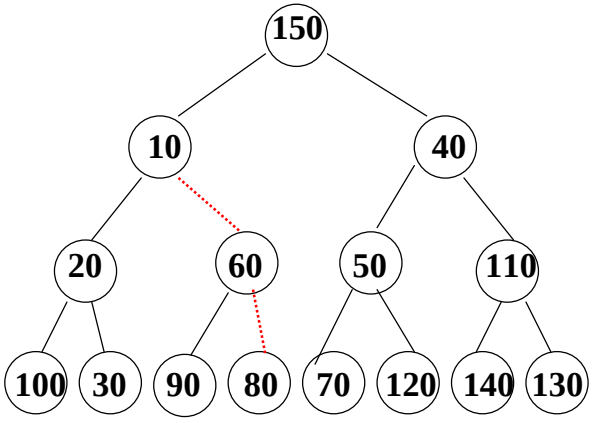
After *percolateDown* (5)



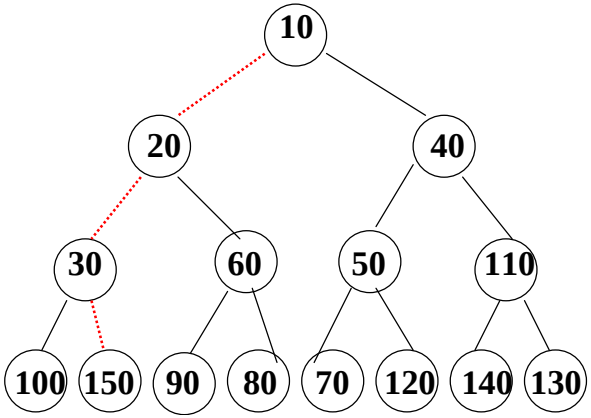
After *percolateDown* (4)



After *percolateDown* (3)



After *percolateDown* (2)



After *percolateDown* (1)

Complexity of buildHeap?

- Analysis
  - percolateDown for n/2 keys
  - Each key takes up to O(logn) cost
  - Thus, the total cost of BuildHeap is O(nlogn)

Is this upper bound tight?

- Notice
  - At most n/4 nodes need to percolate down 1 level
    at most n/8 nodes need to percolate down 2 levels
    at most n/16 nodes need to percolate down 3 levels

$$\cdots 1\frac{n}{4}+2\frac{n}{8}+3\frac{n}{16}+\ldots=\sum_{i=1}^{\log n} i\frac{n}{2^{i+1}}=$$

$$\frac{n}{2}\sum_{i=1}^{\log n}\frac{i}{2^i}\approx\frac{n}{2}(2)=n$$

Conclusion: O(n)

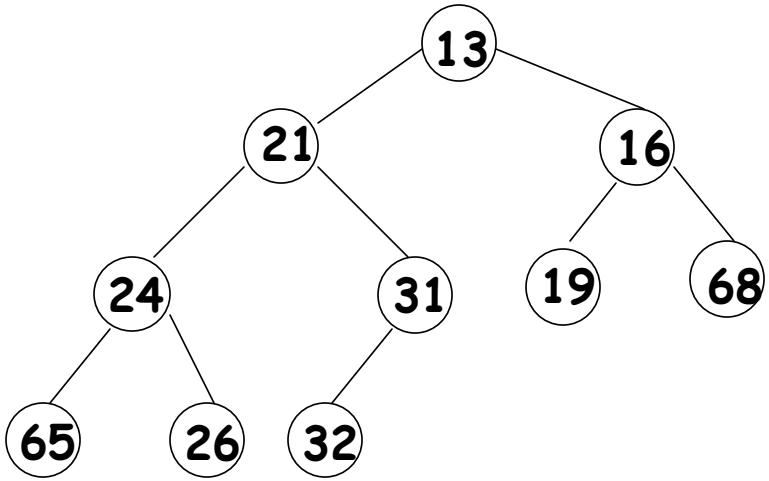
What if we invoke percolateDown (*i*) for i starting from 1 to N/2 ?

Exercise

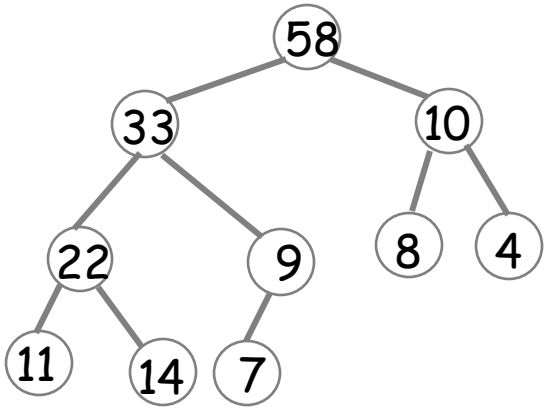
- Given an array of elements A[1…8] = [4, 1, 3, 2, 16, 9, 10, 14], build a heap for it
  - Show the steps one by one
  - Draw some figures

Variants of heap

- Min-heap
  - The key present at the root node must be less than or equal among the keys present at all of its children
  - The same property must be recursively true for all sub-trees



- Max-heap
  - The key present at the root node must be larger than or equal among the keys present at all of its children
  - The same property must be recursively true for all sub-trees

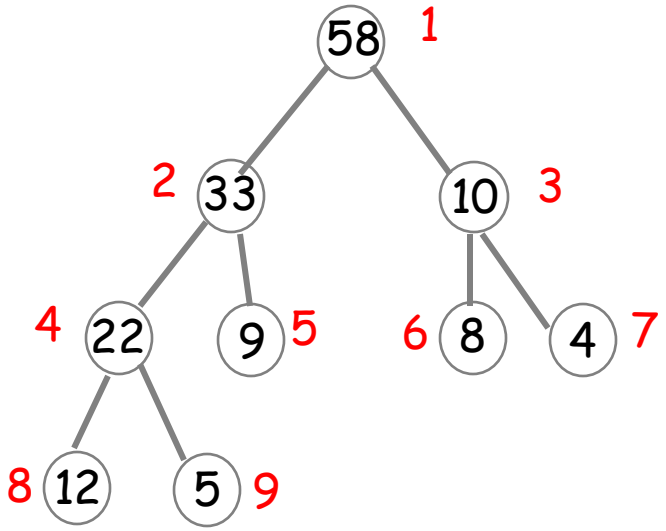


HeapSort

Complexity?

- Sorting using a max-heap
  - To sort an array arr, we first create a max-heap H with a capacity of arr.length+1
  - Then, we repeatedly delete the max element from the max-heap until it becomes empty

4	5	8	9	10	12	22	33	58



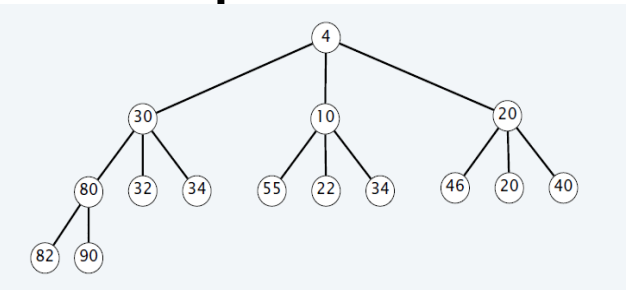
10 classic sorting algorithms

Sorting algorithm	Stability	Time cost			Extra space cost
		Best	Average	Worst	
Bubble sort	√	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Insertion sort	√	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Selection sort	×	O(n)	$O(n^2)$	$O(n^2)$	O(1)
MergeSort	√	O(nlogn)	O(nlogn)	O(nlogn)	O(n)
HeapSort	×	O(nlogn)	O(nlogn)	O(nlogn)	O(1)
QuickSort	×	O(nlogn)	O(nlogn)	$O(n^2)$	O(1)
ShellSort	×	O(n)	$O(n^{1.3})$	$O(n^2)$	O(1)
CountingSort	√	O(n+k)	O(n+k)	O(n+k)	O(k)
BucketSort	√	O(n)	O(n+k)	$O(n^2)$	O(k)
RadixSort	√	O(nk)	O(nk)	O(nk)	O(n)

Stable sorting: if two objects with equal keys appear in the same order in sorted output, as they appear in the input array

6.5 d-Heaps

- d-heap is exactly like a binary heap except that all nodes have d children
  - A binary heap is also called *2-heap*.

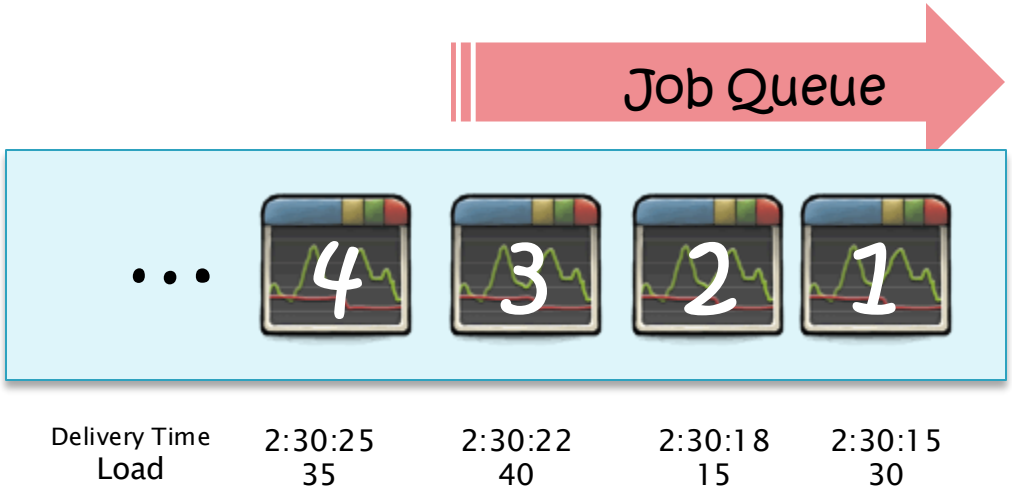


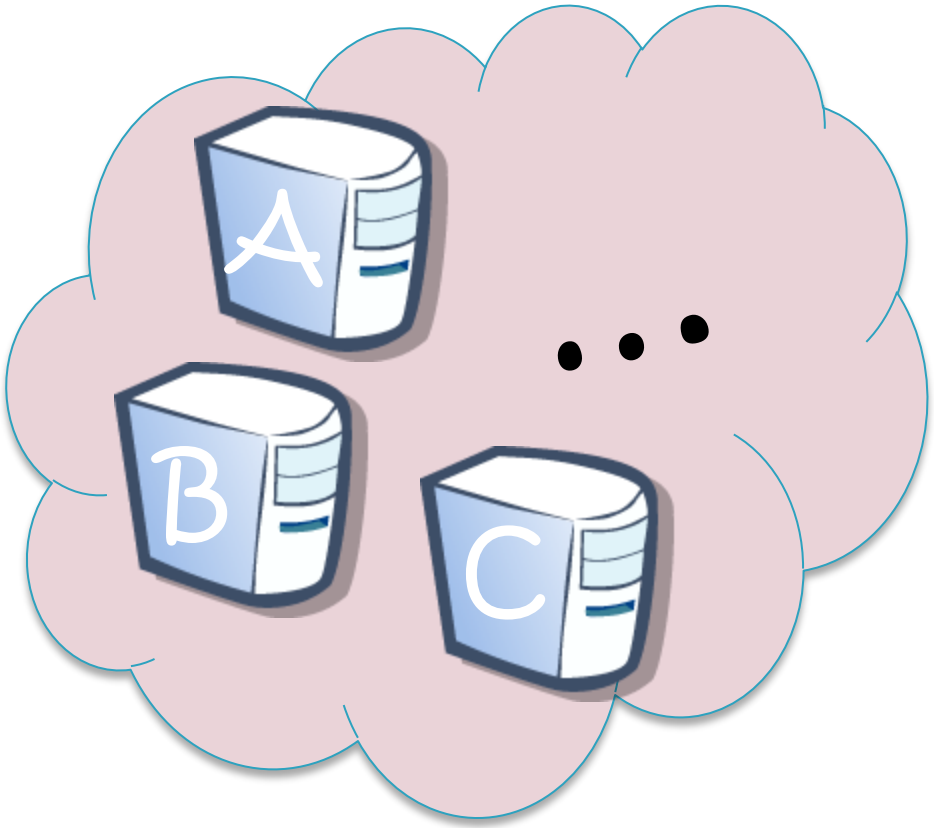
- A d-heap is much shallower than a binary heap
  - Insert operation: $O(\log_d N)$;
  - DeleteMin operation: $O(d\log_d N)$;
  - Useful in some applications where #INSERT >= #DeleteMin
  - In practice, 4-heap outperforms 2-heap.

Load Balancing

- Assign jobs to servers based on the loading of the servers
- Aims to
  - optimize resource use
  - Maximize throughput
  - Minimize response time
  - Etc.

Distributing Jobs to Servers



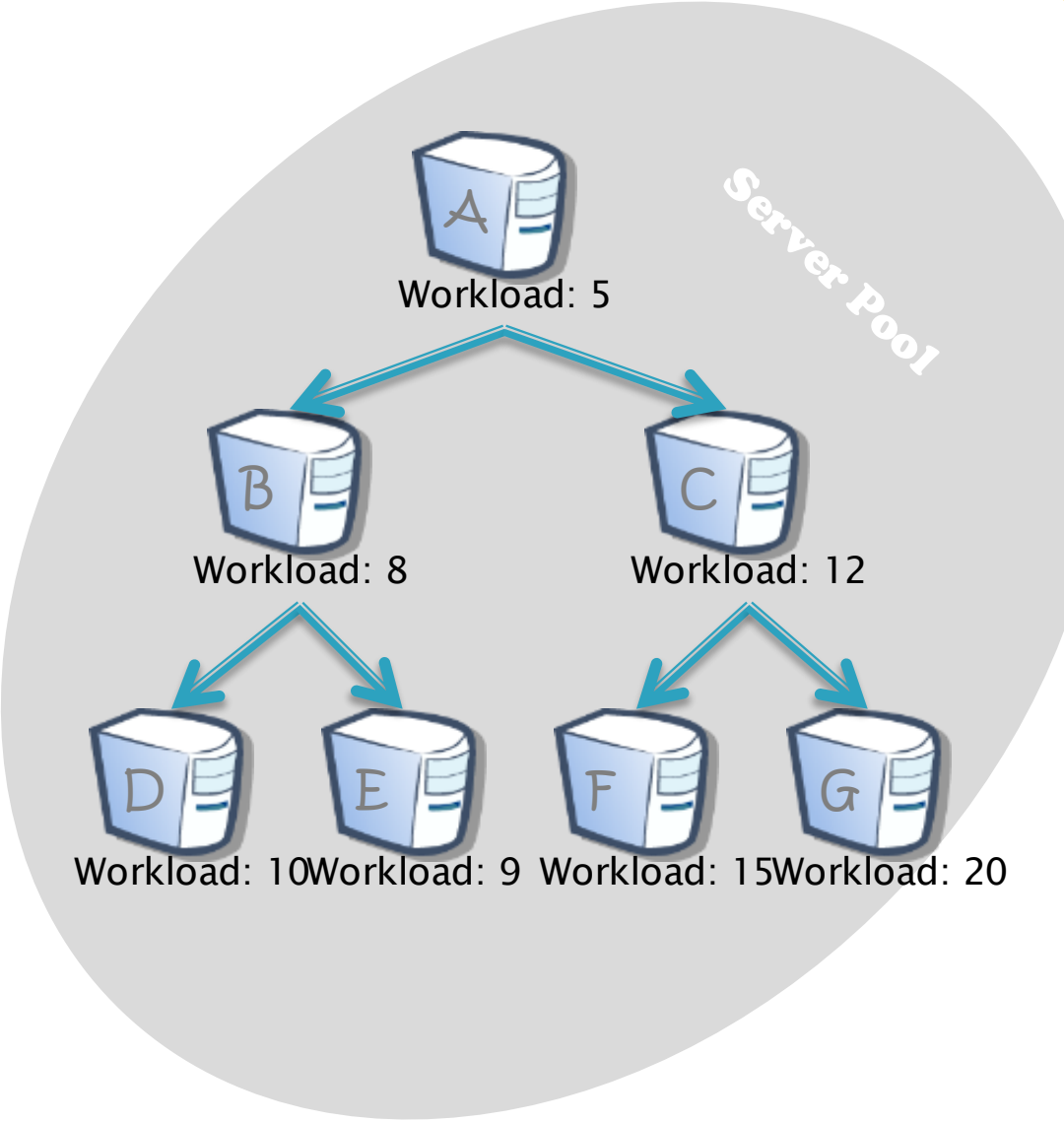


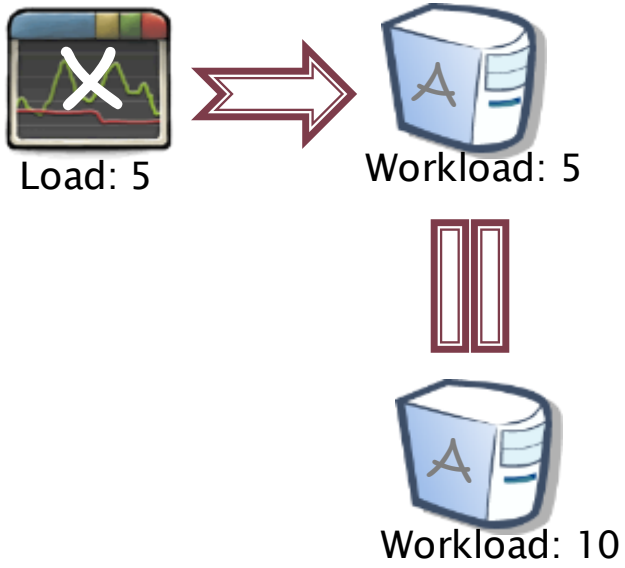

Load: 5

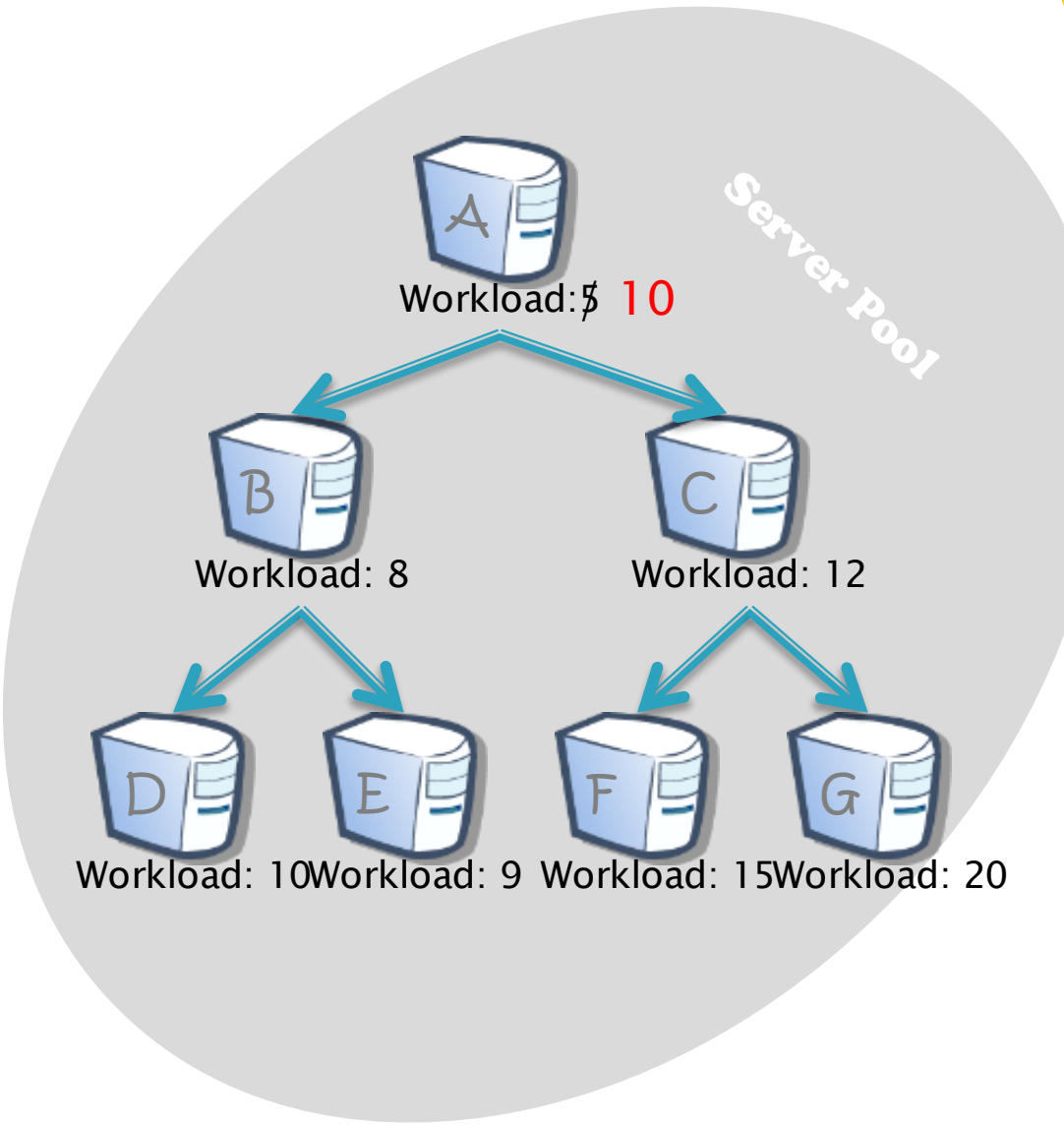


Workload: 5

1. A job delivers
2. Need a server to serve it
3. Server A's workload is lowest

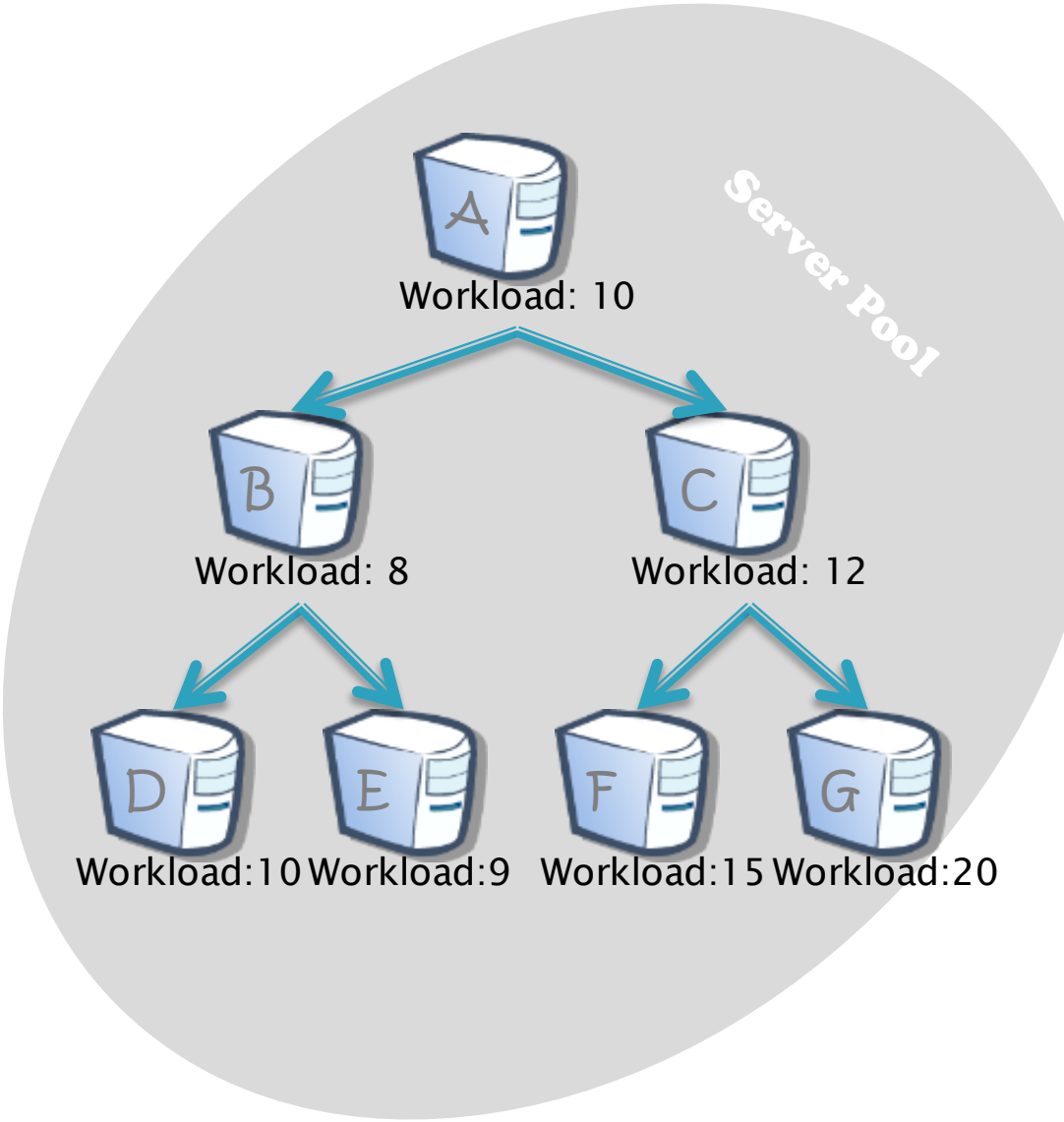




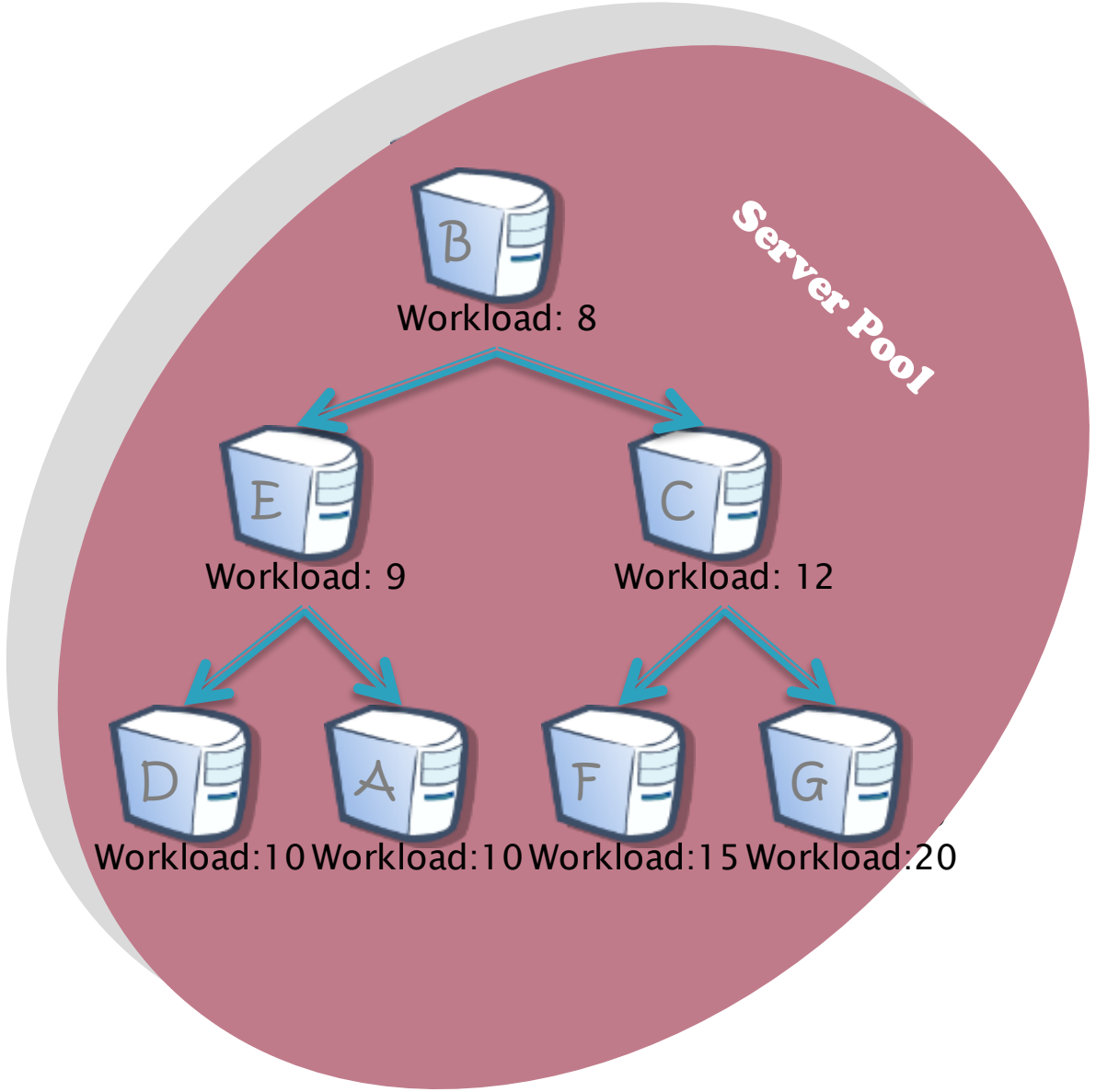


- Server A accepts the job, its workload increased
- Now the servers is not listed in the order property of min heap

- What operation the load balancer should do for maintaining the order property of min heap?



- Bubble-down for the server A



LoadBalancer, itself, is a min binary heap

Initially, all servers are available in the heap

Get a server with lowest workload

Maintain the order property

```
public class LoadBalancer {
        Server[] servers;
        int numOfServers = 0;

        public LoadBalancer(int size) {
                numOfServers = size;
                servers = new Server[size + 1];

                for (int i=1; i<size + 1; i++)
                        servers[i] = new Server();
        }

        public Server getServer() {
                return findMin();
        }

        public void update() {
                bubbleDown(1);
        }

        ... //other methods

}
```

```java
class Server {
    static int counter = 0;

    int id;
    List<Job> jobs;
    int workload = 0;

    public Server() {
        id = counter++;
        jobs = new ArrayList<Job>();
    }

    public void assign(Job j) {
        jobs.add(j);
        workload += j.load;
    }

    public void printJobs() {
        for (Job j: jobs)
            System.out.printf("[ID: %d, Load: %d] ",
            j.id, j.load);
    }
}
```

```java
class Job {
    static int counter = 0;
    int id;
    int load;

    public Job(int load) {
        id = counter++;
        this.load = load;
    }
}
```